L'API DELLE CARTUCCE

API dei giochi bm, volume 3: 3D

Modelli, camere, luci, scheletri, la GPU e i mondi di collisione: tutto ciò che serve a un gioco per disegnare e muovere cose in 3D.

Indice

Come funziona il 3D in bm **4**

Mesh e modelli 4

Chi disegna: la GPU o l'ARM 4

I limiti da conoscere 4

L'ordine dentro un frame 4

Tutorial: un modello sullo schermo **6**

Mesh e modelli **8**

Luce precalcolata 8

Disegno e bit di materiale **9**

I flag di draw3d 9

Bit di materiale 9

Camera, luce e nebbia **11**

Scheletri e animazione **12**

Effetti, etichette e 2D sopra il 3D **13**

Visibilità **14**

La GPU: gpu3d e le sue opzioni **15**

Impostor 15

Cosa fa anche la GPU 16

Anti-aliasing 16

Un frame dopo 16

Mondi di collisione **17**

Terreno di mappa 3D **18**

Luce per le scene 2D **19**

Luce 19

Luce a livelli 19

La libreria n8 20

Consigli sulle prestazioni 21

Punti aperti 22

LEGGIMI

Cos'è. Il volume 3 dell'API dei giochi bm. Copre ogni funzione 3D che una cartuccia può chiamare, con le luci e i mondi di collisione che le accompagnano.

Come si usa. Leggi una volta il primo capitolo, segui il tutorial per portare un modello sullo schermo, poi tieni gli altri capitoli come riferimento. Ogni tabella dà la funzione con i suoi argomenti e cosa fa.

Cosa manca. Ogni lacuna è segnata con un'etichetta evidenziata DA COMPLETARE; nulla è inventato.

Come funziona il 3D in bm

Questo capitolo ti dice cosa succede tra il tuo `_draw` e i pixel sullo schermo.

Mesh e modelli

Un oggetto 3D è una **mesh**: una lista di vertici e di facce triangolari. Una mesh si ottiene in tre modi.

- La costruisci da tabelle di numeri con `mesh()`.
- Ne chiedi una già pronta con `mesh_sphere()` o `mesh_cube()`.
- Carichi un **modello della cartuccia** con `model()`. I modelli si fanno con bm Studio (sezione MESH) e hanno come texture lo sprite sheet della cartuccia. Un modello con scheletro viene da bm Animator e si anima con `animate()`.

Una mesh si disegna con `draw3d()`, da una camera che posizioni con `camera3d()`.

Chi disegna: la GPU o l'ARM

La GPU del Pi (V3D) disegna i triangoli. Con l'impostazione *Settings* > *Graphics* > *3D of the games* su `ARM` (`gpu3d=0` in `bm/config.txt`), e in QEMU, li disegna l'ARM. Le funzioni sono le stesse, quindi una cartuccia non cambia.

- L'ARM trasforma, illumina e ritaglia sempre i triangoli.
- La GPU riempie i pixel con uno z-buffer a 24 bit, ombreggia senza dithering e legge le texture con il texel più vicino.
- Se la GPU non risponde, il kernel torna da solo all'ARM e lo scrive nel log.
- `stat(9)` vale 1 quando il 3D lo fa la GPU.

I limiti da conoscere

- Una mesh ha fino a 65535 vertici e 65535 facce (4096 e 16384 prima di bm3d 5.0).
- I triangoli che attraversano il piano vicino alla camera sono tagliati, non scartati: pavimenti e oggetti grandi restano interi anche quando passano accanto alla camera.
- Nei giochi 256x256 la GPU non disegna (scrive pagine intere): disegna l'ARM.
- Lo z-buffer della GPU riparte da zero a ogni frame, anche senza `zclear()`.
- Le facce **con texture e screen door** insieme la GPU non le sa disegnare: alla prima la cartuccia passa all'ARM per il resto dell'esecuzione (una riga nel log; il frame misto non viene mostrato).

L'ordine dentro un frame

Il 3D in attesa viene disegnato prima di ogni disegno 2D che lo segue, prima di `pget` e `sset`, e alla fine del frame. Disegna prima tutto il 3D e poi l'HUD. Se una cartuccia disegna altro 3D dopo il 2D nello stesso

frame, dal frame successivo (non dal primo) la GPU conserva lo z-buffer tra le due parti: circa 1 MB di memoria scritto e riletto a ogni frame.

Tutorial: un modello sullo schermo

Mostrerai un modello della tua cartuccia, illuminato e in rotazione. Parti da una cartuccia che ha già almeno un modello fatto in bm Studio (sezione MESH) e che si apre in bm Code.

1. In `_init` carica il modello: `m = model(1)`. I modelli si contano da 1. Restituisce `nil` se il modello non c'è, quindi controlla il risultato. Fallo solo in `_init`: ogni chiamata a `model()` costruisce una nuova mesh.
2. In `_init` chiedi il box attorno al modello con `bounds3d(m)`. Restituisce `x0, y0, z0, x1, y1, z1`. Usa `(y0 + y1) / 2` per abbassare il modello in modo che il suo centro stia all'altezza 0.
3. Posiziona la camera una sola volta, in `_init`: `camera3d(0, 0, -6)`. La camera predefinita sta a z = -5 con un campo visivo di 60 gradi.
4. Imposta la luce, una volta: `light3d(0.5, 1, -0.5, 0.3)`. I primi tre numeri sono la direzione della luce e l'ultimo è la luce ambiente (da 0 a 1).
5. In `_draw` pulisci lo schermo con `cls(0)` e poi lo z-buffer con `zclear()`. Fallo a ogni frame, prima di `draw3d`.
6. Sempre in `_draw`, disegna il modello con `draw3d(m, 0, -cy, 0, 0, ang, 0, 1, 4)`: la posizione, poi gli angoli `rx, ry, rz`, poi la scala, poi i flag. Il flag 4 ombreggia la luce in modo morbido.
7. Aggiungi `ang = ang + 0.02` alla fine di `_draw`, così il modello gira un poco a ogni frame.
8. Disegna il testo del gioco (l'HUD) dopo `draw3d`, con `print`.
9. Avvia la cartuccia. Dovresti vedere il modello illuminato da un lato e in rotazione. Se non vedi nulla, il modello può essere dietro la camera o troppo grande. Un modo per scoprirlo: stampa i valori di `bounds3d` e allontana la camera.

[DA COMPLETARE: shots/api-3d-model.png]

```
local m, cy, ang = nil, 0, 0

function _init()
  m = model(1)
  local x0, y0, z0, x1, y1, z1 = bounds3d(m)
  cy = (y0 + y1) / 2
  camera3d(0, 0, -6)
  light3d(0.5, 1, -0.5, 0.3)
end

function _draw()
  cls(0)
  zclear()
  draw3d(m, 0, -cy, 0, 0, ang, 0, 1, 4)
  print("il mio modello", 4, 4, 0xFFFFFF)
  ang = ang + 0.02
end
```

PROVA

Cambia i flag da 4 a 2 e riavvia: il modello perde la luce e mostra colori piatti. Poi prova 4 + 128, che disegna anche le facce con texture come colori piatti.

Gli angoli `rx, ry, rz` di `draw3d` sono in radianti. Anche `yaw` e `pitch` della camera sono in radianti; `fov` è in gradi.

Mesh e modelli

Funzione	Descrizione
`mesh(v, f, [uv])`	una mesh da tabelle (fino a 65535 vertici e 65535 facce: 4096 e 16384 prima di bm3d 5.0): `v` = {x,y,z, x,y,z, ...}, `f` = {a,b,c,colour, ...} (indici da 1; una faccia si vede dal lato in cui i suoi vertici appaiono in senso **orario**). Con `uv` (6 numeri per faccia: u,v dei tre vertici in pixel dello sprite sheet) le facce con colore `-1` hanno la **texture** dello sprite sheet (corretta in prospettiva, i pixel trasparenti restano vuoti). Il colore può avere i **bit di materiale** (vedi "Disegno e bit di materiale")
`mesh_sphere([rings, segments, c1, c2])`	una sfera già pronta. Valori predefiniti: `rings` 8, `segments` 16, `c1` 0xFFFFFF, `c2` 0xC0C0C0. Non c'è il raggio: scalala con `draw3d`
`mesh_cube([c])`	un cubo già pronto. `c` vale 0xFFFFFF in modo predefinito
`model(name) / model(n)`	un **modello 3D della cartuccia** (fatto con bm Studio, sezione MESH) come mesh, con la texture dello sprite sheet; `n` si conta da 1; `nil` se non c'è. Ogni chiamata costruisce una nuova mesh: falla in `_init`
`models()`	i nomi dei modelli della cartuccia, in ordine (`{}` se non ne ha)
`bounds3d(m)`	`x0, y0, z0, x1, y1, z1` : il box attorno ai vertici di una mesh, nelle sue coordinate (prima di spostarla, ruotarla e scalarla con `draw3d`): per centrarla, per le collisioni

Luce precalcolata

Un modello con **luce precalcolata** (il bit "lit" della sezione MESH: la luce di ogni angolo delle facce, fatta da uno script) si disegna morbido con quella luce. Non ha sole né cielo, ma ha le lampade (`lamp3d`) e la nebbia. Costa meno della luce calcolata. Le sue facce con texture (finestre, insegne) hanno una luce per faccia, colorata, e la nebbia.

```
local names = models()          -- {} se la cartuccia non ha modelli
local house = model("house")    -- per nome, oppure model(1) per numero
```

Disegno e bit di materiale

Funzione	Descrizione
`draw3d(m, x, y, z, [rx, ry, rz, scale, flags])`	disegna una mesh con z-buffer e luce per faccia. I `flags` sono nella tabella qui sotto
`zclear()`	pulisce lo z-buffer (a ogni frame, prima di `draw3d`)

I flag di draw3d

I flag si sommano (per esempio 4 + 128).

Flag	Effetto
1	niente z-buffer (né test né scrittura): pavimenti e sfondi disegnati per primi, più veloce
2	niente luce (colori piatti)
4	**smooth** (Gouraud): luce calcolata sui vertici e sfumata sulla faccia, con dithering. Facce che condividono gli stessi indici di vertice sembrano una superficie curva; per gli spigoli vivi usa vertici separati o il bit "flat"
8	l'**ombra** della mesh sul piano `y` del punto (lungo il sole): scurisce ciò che c'è già, la mesh non viene disegnata
16, 32, 48	**livello di dettaglio** 2, 1, 0 (solo le facce di quel livello, vedi i bit di materiale); senza: 3, tutte
64	**davanti** (armi e braccia in prima persona): lo z-buffer sotto viene azzerato, profondità precise da 0.1 unità
128	**texture piatte**: le facce con texture sono disegnate come colori piatti, ognuno la media dei suoi texel. I texel trasparenti sono saltati, e una faccia di soli buchi è grigia. La mesh, le ossa e l'animazione sono le stesse; le facce illuminate conservano la loro luce, le altre facce non cambiano. Sulla GPU una mesh così va triangolo per triangolo, un po' più lenta. I colori sono conservati per mesh e per versione della texture (bm3d 7.0)

Bit di materiale

Questi bit stanno nel colore di una faccia, in `mesh()` e nei modelli. 0 è la faccia normale.

Bit	Valore	Effetto
30	`0x40000000`	**emissiva**: un colore piatto, senza luce (luci, schermi, energia)
29	`0x20000000`	**lucida**: il riflesso del sole (`shine3d`)

Bit	Valore	Effetto
28	`0x10000000`	**screen door**: un pixel sì e uno no, ciò che sta dietro si vede (scudi, vetri)
27	`0x08000000`	**piatta**: anche in un disegno smooth (flag 4) prende la luce del suo piano; gli spigoli vivi possono condividere i vertici
24-26		**livello di dettaglio**: i bit 24-25 un livello `k` (0-3); col bit 26 a 0 la faccia si vede da `k` in su (un dettaglio), a 1 sotto `k` (una versione semplice)

```
local SHIELD = 0x40C8FF | 0x40000000 | 0x10000000   -- azzurro, emissivo, screen door
local DETAIL = 0xFFFFFF | 0x02000000                -- bianco, solo dal dettaglio 2 in su
```

ATTENZIONE

Le facce che sono insieme con texture e screen door fanno rinunciare la GPU: la cartuccia passa all'ARM per il resto dell'esecuzione. Usa le due cose su facce diverse.

Camera, luce e nebbia

Funzione	Descrizione
`camera3d(x, y, z, [yaw, pitch, fov, roll])`	la camera (predefinita a z = -5, fov 60 gradi); `roll` inclina la vista (radianti)
`light3d(x, y, z, [ambient])`	la direzione della luce e la luce ambiente (0-1)
`sky3d(sun, sky, ground)`	colori (0xRRGGBB) della luce del sole e della luce ambiente che viene da sopra e da sotto (le facce rivolte in su prendono il cielo, quelle rivolte in giù il suolo); `sky3d()` torna al bianco
`shine3d(spec, exponent, rim)`	i riflessi del sole sulle facce lucide (`spec` 0-2, `exponent` 4-64: più alto, più piccolo) e la luce sul bordo delle forme (`rim` 0-1)
`fog3d(colour, near, far)`	nebbia: le facce sfumano nel colore tra le due distanze; `fog3d()` la toglie
`lamp3d(i, x, y, z, radius, [k, colour])`	luce puntiforme `i` (1-4): le facce con il centro entro `radius` si illuminano, fino a `k` in più (predefinito 1) al centro, del `colour` dato (bianco se manca); `lamp3d(i)` la spegne, `lamp3d()` tutte. Con `light3d` a una luce ambiente bassa fa scene buie con lanterne
`shadow3d(style)`	le ombre di `draw3d` con flag 8: 0 scurisce (predefinito), 1 nero con dithering (senza leggere lo schermo)
`textures3d([on])`	`textures3d(false)` fa comportare ogni `draw3d` come con il flag 128 (facce con texture come colori piatti, il loro texel medio); `textures3d(true)` torna alle texture (predefinito). Restituisce lo stato attuale. Per un'opzione "piatto / con texture" in un gioco (bm3d 7.0)

Sulla GPU le ombre sono sempre nero con dithering (stile 1 di `shadow3d`), sopra le cose già disegnate grazie allo z-buffer.

```
camera3d(0, 2, -8)
light3d(0.3, 1, -0.4, 0.15)          -- ambiente basso: una scena buia
sky3d(0xFFF0D0, 0x6080C0, 0x302010)
fog3d(0x101820, 10, 40)
lamp3d(1, lx, ly, lz, 6, 1, 0xFFB060)  -- una lanterna
```

Scheletri e animazione

Un modello con scheletro viene da bm Animator. Le ossa muovono i vertici mentre la mesh viene disegnata (skinning rigido), quindi `animate` costa solo le ossa.

Funzione	Descrizione
`animate(m, [anim, t, anim2, t2, k, bone])`	**animazione scheletrica**: il modello prende la posa dell'animazione `anim` (nome o numero) al tempo `t` in secondi (in ciclo, se l'animazione è in ciclo). Con `anim2, t2` miscela due animazioni (`k` da 0, solo la prima, a 1, solo la seconda: per passare dall'una all'altra). Con `bone` la seconda vale solo per quell'osso e quelli sotto di lui (un torso che spara su gambe che corrono). Senza animazione: la posa di riposo. Restituisce la lunghezza dell'animazione. Errore se la mesh non ha scheletro o l'animazione non c'è
`bone_turn(m, bone, [rx, ry, rz])`	da ora ogni `animate` ruota anche l'osso di questi angoli (radianti, x poi y poi z, nel sistema del genitore) sopra l'animazione: mirare su e giù, gambe che seguono la direzione di marcia. `bone_turn(m, bone)` lo toglie
`bones3d(m)`	i nomi delle ossa dello scheletro, in ordine
`clips(m)`	le animazioni di un modello: `{ {name=, length=, loop=}, ... }` (`{}` senza scheletro)
`bone3d(m, bone)`	`x, y, z, cx, cy, cz`: dove stanno la testa e la coda di un osso (nome o numero) nell'ultima posa, nelle coordinate del modello (come `bounds3d`); `nil` se l'osso non c'è. Per attaccare oggetti alle mani (la testa), alla punta di una spada (la coda), luci, effetti
`hit3d(m, x, y, z, ry, scale, ox, oy, oz, dx, dy, dz, [maxd])`	`t, bone`: il raggio da `o` lungo `d` contro le ossa della mesh nell'ultima posa, disegnata in (x, y, z) ruotata di `ry` e scalata. Ogni osso è una capsula dalla testa alla coda larga quanto i suoi vertici. Il colpo più vicino entro `maxd` (`t` in unità di `d`), oppure `nil`: hitbox che seguono l'animazione (colpo alla testa: osso `"head"`)

```
-- in _draw: il villico cammina, poi sfuma in un saluto
local len = animate(npc, "walk", t)
animate(npc, "walk", t, "wave", t, k)      -- k da 0 (cammina) a 1 (saluta)
local hx, hy, hz = bone3d(npc, "hand_r")   -- una luce in mano
```

[DA COMPLETARE: i veri nomi di animazioni e ossa di un modello fornito, per l'esempio qui sopra (`"walk"`, `"wave"`, `"hand_r"` sono solo forme di nomi)]

Effetti, etichette e 2D sopra il 3D

Funzione	Descrizione
`point3d(x, y, z, radius, colour, [flags])`	un punto tondo di quel raggio nel mondo, dietro le cose più vicine (non scrive lo z-buffer): particelle, scintille, proiettili. `flags` 1 = un pixel sì e uno no. Restituisce i pixel
`line3d(x0, y0, z0, x1, y1, z1, colour, [width, flags])`	una linea 3D, tagliata dal piano vicino e nascosta dalle cose più vicine: scie traccianti, raggi
`sprite3d(sx, sy, sw, sh, x, y, z, width, [flags])`	un rettangolo dello sprite sheet rivolto verso la camera, largo `width` unità nel mondo, nascosto dalle cose più vicine: esplosioni, fumo, icone sopra i personaggi
`project3d(x, y, z)`	un punto del mondo in `sx, sy, depth` sullo schermo (`nil` se è dietro la camera): per disegnare in 2D cose allineate al 3D (orizzonte, mirini, etichette)

La GPU disegna anche le ombre, questi effetti (`point3d` , `line3d` , `sprite3d`), le facce screen door e le texture dei modelli con luce precalcolata (luce e nebbia sfumate negli angoli, anche con le `lamp3d`).

```
local sx, sy = project3d(ex, ey + 2, ez)    -- sopra un nemico
if sx then print("goblin", sx - 12, sy, 7) end
```

Visibilità

Funzione	Descrizione
`visible3d(x, y, z, r)`	`false` se una sfera (centro, raggio) non si può vedere: dietro la camera, oltre un bordo dello schermo, oppure (dopo `pvs3d`) su pezzi della mappa che la cella della camera non vede. Per saltare personaggi ed effetti nascosti dai muri prima di disegnarli (bm3d 5.5)
`pvs3d(t)`	la visibilità precalcolata della mappa per `visible3d` (vedi sotto). `pvs3d()` la dimentica

La tabella `t` di `pvs3d` è `{ x0, z0, cell, nx, nz, max_y, sets, boxes }`.

- È una griglia di `nx` per `nz` celle di lato `cell` sul terreno, a partire da (`x0`, `z0`).
- `sets` è una stringa con un byte per cella (riga per riga lungo x). Ogni byte è il numero (da 1) di un pezzo visto da quella cella.
- `boxes` ha sei numeri per pezzo: `x0 y0 z0 x1 y1 z1`.
- Con la camera sopra `max_y` o fuori dalla griglia conta solo la vista. Una sfera fuori da ogni pezzo si vede.

```
if visible3d(ex, ey, ez, 1.5) then
  draw3d(enemy, ex, ey, ez)
end
```

La GPU: gpu3d e le sue opzioni

`gpu3d([on, aa, vs, queue, impostor, later])` restituisce `on, aa, vs, version, queue, impostor, later`. Con gli argomenti cambia la scelta per questa cartuccia (per esempio un menu "3D: GPU / GPU + VS / GPU + AA / ARM" nel gioco). Alla fine dell'esecuzione torna a quella delle impostazioni.

Argomento	Significato
`on`	se la GPU disegna il 3D. Resta `false` se non c'è la GPU (QEMU, `gpu3d=0`) e nei giochi 256x256 (la GPU scrive pagine intere)
`aa`	se con MSAA 4x. Resta `false` se l'MSAA non si può usare
`vs`	se il vertex shader della GPU posiziona i vertici dei modelli: `false`, `1` (lo scenario: modelli senza luce o con luce negli angoli) oppure `2` (tutti, anche quelli illuminati dal sole e con ossa; `true` vale 2). Resta `false` se il test all'avvio non ha visto funzionare il vertex shader (e il suo clipping)
`queue`	se il lavoro della GPU parte senza aspettarlo e l'`_update` del frame successivo gira nel frattempo (dove il test all'avvio l'ha visto funzionare). Il 2D disegnato dopo il 3D (l'HUD) e quello dell'`_update` vengono registrati e vanno sulla pagina quando la GPU ha finito, nello stesso ordine: il risultato è lo stesso. Un `_update` che disegna 3D o legge lo schermo con `pget` torna a girare dopo il frame. `2`: due lavori in volo. `true` chiede il livello proprio della console (`gpu3d_queue` in Settings, 1 o 2); `false` nessuno. Il quinto valore restituito è il livello in vigore (`1`, `2`) oppure `false`
`impostor`	spento in modo predefinito. Un modello il cui box sullo schermo è al massimo di quel numero di pixel (2-64) di uno schermo di 1080 righe viene disegnato come un solo quad rivolto alla camera nel suo colore medio, illuminato per intero (2 triangoli invece di tutte le sue facce), dalla GPU o dall'ARM allo stesso modo. `false` o `0` lo spegne
`later`	il 6° argomento, solo con `queue 2`; mai in un gioco giocato `online`. Ogni frame è mostrato un frame dopo, così `_update` e `_draw` del frame successivo girano mentre la GPU lo disegna: le stesse immagini, un frame dopo (vedi "Un frame dopo")

Il valore `version` è la versione dei driver 3D che questa scelta riproduce: `"0.2"` l'ARM, `"2.1"`, `"3.0"`, `"3.4"`, `"4.1"`, `"6.9"` un frame dopo (`docs/DRIVERS.md`).

Impostor

- Lo schermo del gioco è scalato all'altezza della TV. A 640x360 il limite è un terzo di pixel del gioco, a 720p due terzi, così un modello ha la stessa dimensione quando viene scelto, qualunque sia la risoluzione.

- Anche un modello con ossa è disegnato come impostor, in base al box della sua posa (dove `animate()` ha messo la parte di ogni osso). Un modello animato fuori dalla vista viene allora anche saltato, come uno senza ossa.
- Si può dare con `on` `nil` : `gpu3d(nil, nil, nil, nil, 16)` .
- La scelta dipende solo dalla camera, dal modello, dalla sua posa e dalla dimensione dello schermo del gioco, quindi è la stessa su ogni console di una partita.
- Il valore predefinito della console è `r3d_impostor` in `bm/config.txt` (*Settings* > *Screen and sound* > *3D impostors*).

Cosa fa anche la GPU

La GPU disegna le ombre (`draw3d` con flag 8), gli effetti, le facce screen door e le texture dei modelli con luce precalcolata. Un frame che inizia con `cls()` costa meno alla GPU: i tile partono dal colore di `cls` invece di rileggere la pagina. Le texture con lati multipli di 32 (sprite sheet 128x128, 256x256, ...) vanno alla GPU in formato T, il formato a tile della sua cache, più veloce da leggere.

Anti-aliasing

Settings > *Graphics* > *3D anti-aliasing: 4x* (`gpu3d_aa=1` in `bm/config.txt`) fa disegnare il 3D alla GPU con MSAA 4x: quattro campioni per pixel, la media alla fine del tile, bordi dei triangoli senza scalini. Nelle cartucce non cambia nulla.

- Vale solo dove la GPU lo regge (altrimenti il test all'avvio dice *4x: not on this GPU*).
- Vale solo nei lavori senza z-buffer conservato: le cartucce che disegnano 3D, poi 2D, poi altro 3D nello stesso frame restano senza anti-aliasing (lo z-buffer a 4 campioni non si può salvare).
- Se il test trova che la GPU non può ricaricare la pagina nei 4 campioni, l'MSAA è usato solo nei frame che iniziano con `cls()` .
- Sull'ARM non c'è anti-aliasing.

Un frame dopo

Settings > *Screen and sound* > *3D frame shown: A frame later* (`gpu3d_later=1` , con la coda dei frame su due lavori, `gpu3d_queue=2` ; da un gioco il sesto argomento di `gpu3d`) mostra ogni frame un frame dopo. Alla fine di `_draw` la GPU avvia il 3D del frame e il frame aspetta, mentre `_update` e `_draw` successivi girano su un'altra pagina. Il 2D disegnato dopo il 3D va sul frame in attesa quando la GPU ha finito, e quel frame è mostrato alla fine del successivo.

- Nelle cartucce non cambia nulla e le immagini sono le stesse, un frame dopo: l'input arriva sullo schermo un frame più tardi che senza la coda.
- Tieni lo stato del gioco in `_update` (come per i giochi online) e disegna l'HUD dopo il 3D. `pget` legge la pagina che si sta disegnando.
- Non è usato nei giochi giocati `online` (lì viene escluso), via RAM (`light_begin` , `dark_begin`) e nel frame di un cambio di `screen()` .
- È spento in modo predefinito finché non viene misurato sul Pi.

Mondi di collisione

Box solidi, raggi e corpi che si muovono scivolando sui muri, in C: molto più veloci che in Lua. Esempio completo: `carts/overbit`. Per la mappa 2D ci sono i flag dei tile (`mflags`) e `lib.move` / `lib.step` di bmlib.

Funzione	Descrizione
`world3d()`	un mondo di collisione vuoto
`world_box(w, x0, y0, z0, x1, y1, z1, [tag])`	un box solido; restituisce il suo numero
`world_ray(w, ox, oy, oz, dx, dy, dz, [maxd, ground])`	`t, nx, ny, nz, box`: il primo punto colpito lungo il raggio (la normale della faccia; box 0 = il suolo `y = 0`, che conta a meno che `ground` sia `false`), oppure `nil`
`world_move(w, x, y, z, r, h, dx, dy, dz, [step, on_ground])`	`x, y, z, flags`: un corpo (piedi in `x, y, z`, raggio `r`, altezza `h`) spostato di `d`, che scivola sui muri e sale gradini fino a `step` (0.45) se era a terra. `flags`: 1 a terra, 2 un muro (4 lungo x, 8 lungo z), 16 un soffitto
`world_floor(w, x, z, y, r, [step])`	l'altezza del pavimento sotto `(x, z)`

```
local w = world3d()
world_box(w, -10, -1, -10, 10, 0, 10)          -- una lastra
local x, y, z, fl = world_move(w, px, py, pz, 0.4, 1.8, vx, vy, vz, 0.45, on_ground)
on_ground = (fl & 1) ~= 0
```

Se `on_ground` manca in `world_move` vale `false`. Il `tag` di `world_box` vale 0 in modo predefinito.

[DA COMPLETARE: a cosa serve `tag` di `world_box`]

Terreno di mappa 3D

Un mondo 3D costruito con i livelli della mappa 2D: l'anteprima 3D dell'editor e i giochi usano queste funzioni. Il terreno viene dai livelli `floor`, `height` e `walls`; i segnaposto vengono dal livello `objects`. Il mondo si disegna a blocchi di 16x16 tile, e si disegnano solo i blocchi che si vedono.

Funzione	Descrizione
`map3d_build([opts])`	costruisce il mondo dai livelli della mappa e lo restituisce. `opts = { floor = "floor", height = "height", walls = "walls", cell = 1.0, step = 0.25 }`: i nomi dei livelli, la dimensione di un tile in unità del mondo e il passo dell'altezza. Ogni campo è facoltativo. Il livello degli oggetti è sempre `objects`
`map3d_draw(w, [flags])`	disegna i blocchi di terreno che si vedono; `flags` è facoltativo (0 se manca)
`map3d_height(w, x, z)`	l'altezza del terreno in `(x, z)`, o `nil` per un buco o un punto fuori dalla mappa
`map3d_blocked(w, x, y, z, r)`	`true` se una sfera con centro `(x, y, z)` e raggio `r` tocca un muro o un dirupo
`map3d_objects(w, [kind])`	la lista degli oggetti `{ kind, x, y, z }`; `kind` filtra la lista
`map3d_size(w)`	`nx, nz`: la dimensione del mondo in unità del mondo

Chiama prima `zclear()` e `camera3d`, poi `map3d_draw`. Se la mappa è vuota, `map3d_build` dà un errore. Per un esempio completo con più livelli di gioco, vedi il manuale *Mappe e livelli*.

Luce per le scene 2D

Queste funzioni non sono 3D: oscurano una scena 2D illuminata da lampade. Dal primo `light_begin()` o `dark_begin()` la cartuccia disegna in RAM invece che direttamente sullo schermo.

Luce

Scene buie illuminate solo da lampade, candele, torce: il disegno del frame viene moltiplicato per una "mappa di luce" calcolata in C (una griglia ogni 4 pixel, interpolata e leggermente con dithering).

Funzione	Descrizione
`light_begin([ambient])`	avvia le luci del frame: `ambient` è il colore della luce di fondo (`0x000000` buio pesto, `0xFFFFFF` nessun effetto)
`light(x, y, radius, colour, [intensity])`	una luce morbida in coordinate del mondo (vale `camera`); più luci si sommano, fino a 2x la luminosità
`light_end()`	applica la luce a tutto ciò che è stato disegnato; ciò che disegni dopo (HUD, testo) resta a piena luce

```
cls(0); map(...); spr(...)                  -- la scena
light_begin(0x0A0A16)                        -- notte blu scuro
light(lx, ly, 50 * (0.95 + math.random() * 0.1), 0xFFB060)   -- un lampione che vibra
light(px, py, 40, 0xFFC888, 0.9)             -- la lanterna del giocatore
light_end()
print("life", 4, 4, 0xFFFFFF)                -- l'HUD non viene oscurato
```

Esempio completo: `carts/hunt` .

Luce a livelli

L'altra luce funziona a livelli: ogni pixel ha un **livello di luce** (0 il più buio) e il suo colore diventa quello che una **tabella di dissolvenza** dà a quel livello. Le lampade fanno anelli concentrici di livelli, dal loro livello al centro fino a 0 al bordo. I bordi degli anelli sono mescolati con un dithering ordinato 4x4; dove due lampade si incontrano vince la più forte. Le tabelle le sceglie la cartuccia: le ombre possono diventare blu notte e i colori vicino alle lampade arancioni, restando sui colori della sua palette. Tutto in C.

Funzione	Descrizione
`fades(tables)`	le tabelle: `{ {colour, l0, l1, ...}, ... }` , per ogni colore della palette cosa diventa al livello 0 (il più buio), 1, ...; tutte le righe hanno lo stesso numero di livelli (2-16, fino a 255 colori). I colori senza tabella sono scalati come la media delle tabelle. Restituisce il numero di livelli
`dark_begin([ambient])`	avvia il frame: ogni pixel al livello `ambient` (predefinito 0); da qui la cartuccia disegna in RAM
`glow(x, y, radius, level, [dither])`	una lampada in coordinate del mondo (vale `camera`): `level` al centro, 0 a `radius` ; `dither` 0-1 (predefinito 0.5) è quanto si mescolano i bordi degli anelli (0 anelli netti, 1 una sfumatura continua con dithering)

Funzione	Descrizione
`dark_end()`	applica i livelli a tutto ciò che è stato disegnato; ciò che disegni dopo (fiamme, scintille, HUD) resta com'è e "brilla"

```
fades(TABLES)                              -- una volta, in _init
cls(0); map(...); spr(...)                 -- la scena a piena luce
dark_begin(1)                              -- notte: livello 1 ovunque
glow(lx, ly, 72 + math.random(2), 6)       -- un lampione
glow(px, py, 28, 3)                        -- la piccola luce attorno al giocatore
dark_end()
spr(FLAME, fx, fy)                         -- le fiamme non vengono oscurate
```

Esempio completo: `carts/yharnam`. Uno piccolo: `SQUARE_CART` in `tests/qemu_test.py` (una lampada su una cartuccia 256x256).

La libreria n8

Ogni cartuccia vede anche la tabella `n8`: la macchina **nano8**, con le funzioni delle cartucce `.p8` (`n8.spr`, `n8.map`, `n8.print`, `n8.peek` e le altre, con argomenti e numeri come le vogliono) e quelle per caricarle, avviarle e mostrarle (`n8.load`, `n8.power`, `n8.buttons`, `n8.blit`, `n8.preview`, `n8.compile`). È fatta per `carts/nano8`; un gioco `.bm` non ne ha bisogno.

Consigli sulle prestazioni

Sono i consigli che il repository dichiara; i numeri vengono dal 3D Bench (*Dev* > *3D Bench*, o il tasto `j` del monitor).

- Disegna prima tutto il 3D, poi il 2D e l'HUD. Il 3D dopo il 2D nello stesso frame fa conservare lo z-buffer alla GPU (circa 1 MB scritto e riletto a ogni frame).
- Inizia il frame con `cls()` : i tile della GPU partono dal suo colore e non rileggono la pagina.
- Disegna prima pavimenti e sfondi con il flag 1 (niente z-buffer): più veloce.
- Usa modelli con luce precalcolata per lo scenario: costano meno della luce calcolata.
- Salta ciò che non si vede con `visible3d` (e `pvs3d` per le mappe) prima di disegnarlo.
- Usa `impostor` per le folle: un modello lontano diventa un solo quad (2 triangoli).
- Costruisci i modelli in `_init` , non in `_draw` : ogni chiamata a `model()` costruisce una nuova mesh.
- Usa texture con lati multipli di 32 (128x128, 256x256): la GPU le legge più velocemente.
- Usa i mondi di collisione invece degli stessi test in Lua: sono in C e molto più veloci.
- Una scena misurata (640x360 EXTREME) aveva un lavoro della GPU di 4,2 ms in un frame di 19,6 ms: era l'ARM a dare il passo. A 1080p una partita giocata è limitata dal riempimento: meno pixel, non meno triangoli, è la leva. Con `stat(16)` vicino al tempo del frame è la GPU a dare il passo; molto sotto, è l'ARM.
- Misura: `stat(16)` è il tempo di lavoro della GPU dell'ultimo frame in ms (0 sull'ARM). Con la coda attiva include l'attesa prima che l'ARM guardi: fidati solo senza la coda.
- Il bench dà un punteggio a ogni driver rispetto a bm3d 2.1 sul Pi Zero W (1000 punti). `make test-b3d` fa un breve controllo sul PC; i millisecondi del PC non contano.

> **NOTA**
> Il 3D Bench misura ogni test con l'ARM e con ogni opzione GPU che la console sa riprodurre, così puoi confrontare i profili.

[DA COMPLETARE: valori in frame al secondo per una cartuccia tipica sul Pi Zero W e sull'RGB30, misurati con il 3D Bench]

Punti aperti

- **Tutorial: un modello sullo schermo**: immagine da fare: shots/api-3d-model.png
- **Scheletri e animazione**: i veri nomi di animazioni e ossa di un modello fornito, per l'esempio qui sopra (`"walk"` , `"wave"` , `"hand_r"` sono solo forme di nomi)
- **Mondi di collisione**: a cosa serve `tag` di `world_box`
- **Consigli sulle prestazioni**: valori in frame al secondo per una cartuccia tipica sul Pi Zero W e sull'RGB30, misurati con il 3D Bench